

(B.1) Uso de memoria en el Driver

Para conocer el uso de memoria en el Driver, podemos hacer uso de `.get("spark.driver.memory")`:

```
In [1]: import os
        from pyspark.sql import SparkSession

        # definir explícitamente las rutas de entorno para macOS
        os.environ['SPARK_HOME'] = '/opt/homebrew/Cellar/apache-spark/4.1.1/libexec'

        spark = SparkSession.builder.appName("memoria driver").getOrCreate()
        configuracion = spark.sparkContext.getConf()
        memoria_driver = configuracion.get("spark.driver.memory")
        print(f"Memoria reservada para el Driver: {memoria_driver}")

        spark.stop()
```

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/03/31 16:52:06 WARN Utils: Your hostname, MacBook-Pro-de-Ivan.local, resolves
to a loopback address: 127.0.0.1; using 192.168.231.120 instead (on interface en
0)
26/03/31 16:52:06 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another a
ddress
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel
(newLevel).
26/03/31 16:52:07 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
Memoria reservada para el Driver: None
```

De forma análoga, la reservada para los workers necesitará de la llamada de `.get("spark.executor.memory")`.

```
In [2]: spark = SparkSession.builder.appName("memoria workers").getOrCreate()

        configuracion = spark.sparkContext.getConf()
        memoria_workers = configuracion.get("spark.executor.memory")
        print(f"Memoria reservada para los Workers: {memoria_workers}")

        spark.stop()
```

Memoria reservada para los Workers: None

(B.2) Workers disponibles

Podemos ver el número de workers dedicados, así como el número de cores totales. El número de workers disponible será, al menos en este caso, el número de cores menos el número de workers dedicados:

```
In [3]: spark = SparkSession.builder.appName("AnalisisWorkersYParticiones").getOrCreate()
```

```

sc = spark.sparkContext

nucleos_totales = sc.defaultParallelism

nodos_activos = sc._jsc.sc().getExecutorMemoryStatus().size() - 1

print("-" * 55)
print(f"Workers (Ejecutores) dedicados : {nodos_activos}")
print(f"Núcleos (Cores) totales      : {nucleos_totales}")
print("-" * 55)

spark.stop()

```

```

-----
Workers (Ejecutores) dedicados : 0
Núcleos (Cores) totales      : 10
-----

```

Es decir, en este caso habrá 10 workers disponibles.

El estándar para el número de particiones de un sistema en computación de altas prestaciones es entre 2 y 4 veces el número de núcleos. Esto implicaría entre 20 y 40 particiones para este caso.

(B.3) Comprobación de homogeneidad

Para comprobar si un RDD es homogéneo usaremos una función trivial de PySpark:

```

In [4]: def es_rdd_homogeneo(rdd):

        if rdd.isEmpty():
            return True

        num_tipos = rdd.map(lambda x: type(x)).distinct().count()

        return num_tipos == 1

```

- Los None son una clase de Python. Por lo tanto, si se "cuela" un None en un RDD de todo int, contaría 2 tipos y el RDD pasaría a ser heterogéneo.
- Por su parte, NaN es de tipo float en Python. Por lo tanto, si el RDD es de tipo float podría tener NaNs sin pasar a ser heterogéneo. Pero esto no pasaría con ints.

Esto es bastante peligroso. A continuación se presentan algunos motivos de por qué es obligatorio tener este problema en cuenta:

- Se supone que Spark está pensado para ejecutar cantidades masivas de datos. Pero usa lazy evaluation, por lo que el problema de tipado no lo detectaría antes de ejecutar, sino que durante la ejecución. Podría destruir ejecuciones que lleven trabajando horas por un error de tipado al final del RDD. Es por eso que es muy importante controlarlos antes de ejecutar.

- Si los RDDs son predecibles y homogéneos, Spark puede usar memoria contigua y serializadores veloces. Sin embargo, de ser heterogéneos, Spark se ve obligado a usar Pickle, el serializador de Python. Esto hunde el rendimiento de la CPU y dispara el consumo de RAM.

(B.4) Particiones desequilibradas.

```
In [5]: spark = SparkSession.builder.appName("DesbalanceoParticiones").getOrCreate()
sc = spark.sparkContext

datos = range(1, 101)
rdd_equilibrado = sc.parallelize(datos, 4)

# glom() para contar cuántos elementos hay en cada partición
conteo_inicial = rdd_equilibrado.glom().map(len).collect()
print(f"Distribución inicial (equilibrada): {conteo_inicial}")
# esto debería ser una distribución equilibrada

# ahora barremos los elementos de los 3 primeros nodos
rdd_desbalanceado = rdd_equilibrado.filter(lambda x: x > 75)

conteo_final = rdd_desbalanceado.glom().map(len).collect()
print(f"Distribución tras el filter (desbalanceada): {conteo_final}")

spark.stop()
```

Distribución inicial (equilibrada): [25, 25, 25, 25]

Distribución tras el filter (desbalanceada): [0, 0, 0, 25]

(B.5) Operaciones de Shuffling

El Shuffling de Spark son las operaciones que implican mover datos entre nodos necesarias para llevar a cabo un programa en ejecución. Se pueden intentar hacer explícitas, pero en realidad Spark toma las decisiones de cómo se hacen. Como es de esperar, esto mata el rendimiento, por lo que conviene evitarlas en la medida de lo posible. Podemos agrupar las transformaciones en tres grupos, dependiendo de lo mucho que penalicen la ejecución o la red:

- Nivel "Cero": `map()`, `flatMap()`, `filter()`, `union()` no penalizan el rendimiento en absoluto por culpa del shuffling. Todos los cálculos ocurren en la memoria local.
- Nivel "Medio": `reduceByKey()` y `aggregateByKey()`: algunos datos necesitan combinarse con otros de otros workers. Sin embargo, todavía están optimizadas; Spark hace que envíen solo lo necesario. Primero hacen las operaciones internas y después las que implican operar con otros nodos.
- Nivel "Crítico":
 - `groupByKey()` agrupa todos los valores asociados a una clave y los "empuja" a un único worker, estén donde estén.

- `repartition(n)` baraja de forma aleatoria entre todos los clúster hasta tener una partición óptima. Es extremadamente ineficiente.
- `join()` y `cogroup()` Spark se ve obligado a mandar todas las filas con la misma clave al nodo en cuestión.
- `sortByKey()` o `sortBy()` en este caso, si bien hace operaciones en local, Spark tendrá que juntar a todos ellos en un mismo worker.

(B.6) Números pseudo-aleatorios con Spark

La API de Spark soporta la generación de números pseudo-aleatorios en su módulo de funciones:

```
In [ ]: import pyspark.sql.functions as F

spark = SparkSession.builder.appName("GeneracionAleatoriaSQL").getOrCreate()

# creamos dataframe vacío de 5 filas vacías
df_base = spark.range(5)

# e insertamos números pseudo-aleatorios tomados de dos distribuciones diferente
df_aleatorio = df_base.withColumn("num_uniforme", F.rand(seed=42)) \
                        .withColumn("num_normal", F.randn(seed=42))

df_aleatorio.show()

spark.stop()
```

```
+---+-----+-----+
| id|      num_uniforme|      num_normal|
+---+-----+-----+
|  0| 0.8017532427858894| 1.1027054481455365|
|  1| 0.7971301351894658| 0.7400395449950132|
|  2| 0.6712601471691363| 0.8951356612564867|
|  3| 0.47952506308000253|-0.04167221574820542|
|  4| 0.05778277112153141|-0.6491629452770356|
+---+-----+-----+
```

Nota: podríamos hacer esto con el Spark tradicional usando RandomRDDs, del módulo de ciencia de datos.

Si generáramos estos números con una librería tradicional de Python, sería complejo orquestar a todos los hilos lanzados desde cada nodo worker para que generen a partir de la misma semilla, lo que impediría que el experimento fuera reproducible. De iniciarlos todos con la misma semilla, sería aún peor, pues todos generarían exactamente la misma secuencia de dígitos, perdiendo esa aleatoriedad que buscábamos. De ahí que la orquestación no pasa por darles a todos la misma semilla, sino que algo mucho más complejo que eso. En suma a todo esto, los números generados con Python no estarían igual de comprimidos que al hacerlo con Spark, pues no lo haríamos desde la MV de

Java. Si estos números viajan por la red de nodos, esto podría ser un problema que afecte al rendimiento del programa.

Para asegurar que la partición que estamos haciendo es diferente al resto de particiones para no generar la misma secuencia, tendríamos que comunicar de alguna forma esos nodos, añadiendo semáforos u otros elementos de control que eviten las condiciones de carrera asegurando a su vez la concurrencia, y esto puede ser inasumible hasta cierto punto. Tal vez se podría garantizar que para un número N de nodos, para una misma semilla, se generen números propios para cada nodo. Esto es, asignar un índice a cada nodo, haciendo que éste solo genere aquellos números cuyo orden en la secuencia sea múltiplo del mismo.

(B.7) Archivos de extensión parquet

A diferencia de formatos tradicionales como CSV o JSON, que almacenan la información fila por fila en texto plano, Apache Parquet utiliza una arquitectura puramente columnar y binaria diseñada para exprimir al máximo el rendimiento en Big Data.

Cuando consultas un CSV, el sistema está obligado a escanear el documento entero de principio a fin, leyendo todas las columnas aunque solo necesites una. Parquet cambia las reglas del juego: agrupa físicamente todos los valores de una misma columna en el disco. Esto permite a motores como Spark aplicar el *column pruning*, es decir, saltar directamente a los datos solicitados ignorando el resto del archivo.

Además, al empaquetar datos del mismo tipo de forma contigua, Parquet aplica algoritmos de compresión extrema y almacena metadatos internos (como valores máximos y mínimos). Esto habilita el *predicate pushdown*, permitiendo al motor descartar bloques enteros de datos antes de siquiera leerlos, ahorrando tiempo y costes masivos.